

OS BIGLAB 汇报

杨凯森 | 2026-05

Biglab B主要成果：

1

Starryos单核编译
Starryos
跑了6h,.....,终于完成了

2

Starryos多核编译Starryos
原先的pipeline
太慢了，换了实验环境后，最后优化到
~400s编译成功

BigLab-A: rCore Tutorial 章节训练

基础内核

tg-rcore-tutorial-ch{1,2}-yks23

裸机启动、no_std/Rust、SBI/QEMU
运行；用户程序加载、trap/syscall
和批处理执行。

核心 OS 机制

tg-rcore-tutorial-ch3/4/5/6-yks23

覆盖多道程序与时间片、地址空间与页表
、进程
fork/exec/wait、文件系统与块设备。

综合与扩展

tg-rcore-tutorial-ch8-yks23
ch{3,4,5}-yks23-t2l10

继续补线程/同步、图形或综合
demo，并对前几章做扩展版本。

形成的能力

把 RISC-V、QEMU、trap、地址空间、进程、文件系统和同步这些基础机制跑通，为后续
StarryOS/TGOSKit 的真实 workload 修复打底。

BigLab-B Task 1: tg-arceos-tutorial 基础练习

完成的 5 个 exercise

exercise-printcolor
exercise-hashmap
exercise-altalloc
exercise-ramfs-rename
exercise-sysmap

能力培养

从基础输出、集合/
哈希表、替代
allocator, 到 RAMFS
rename、系统符号/
地址映射, 逐步补齐
ArceOS 小实验能力。

BigLab-B Task 2 阶段 1: Harness 初版

做法

- 围绕 syscall、busybox 小命令和文件系统行为生成测试用例。(Generator)
- 模型读日志、提出怀疑点、继续补 case 或定位代码。(Debugger)

经验

- 能快速覆盖很多边界输入
- 真实 OS bug
数量不稳定，容易陷入泛化测试和低价值失败。

纯测试挖掘可以作为补充，但是缺乏真实目标使得没有体系的效果

BigLab-B Task 2 阶段 1: 当前Harness

Harness 后来从泛化测试挖掘，收敛到和 StarryOS dev 紧密联动：跑真实任务、读输出、修内核、补测例、提 PR。



自动同步

每天早上 9:00 拉取最新 dev，专门同步分支，减少上游快速演进带来的冲突成本。

自动修复闭环

在 StarryOS 上跑具体任务；一旦发现相关 bug，整理 root cause 与 Test Suite，随 OS 改动提 PR 到 dev。

反馈加速器

每天自动审查工作进展，提出能够加快迭代速度的建议

PR 守护

定时检查 PR CI 状态；未通过则继续收集失败、修复、推送，直到达到可合并状态。

Task 2 PR: 12 个已合入 OS 修复

进程 / 线程 / futex

#692 robust futex cleanup
#693 vfork parent blocking
#878 teardown context

文件系统 / 设备

#695 rsex4 inode bitmap
#800 device full transfer
#844 tmpfs rename exec

SMP / 同步 / 调度

#842 CPU topology
#879 raw mutex handoff
#885 syscall entry snapshot
#926 SMP wakeup progress

网络 ABI

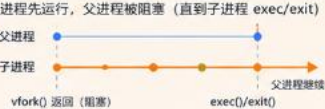
#694 IPv4-mapped IPv6 socket
保持 IPv6 用户态语义，内部复用 IPv4 backend。

验证标准

每个 PR
均包含问题根因、修复范围、测试用例
和 CI 或本地复现证据。

进程创建语义对比：fork / vfork / spawn

三种创建方式在地址空间、执行流程、资源共享与使用场景上的关键差异

	fork()	vfork()	spawn (posix_spawn)
内存语义 地址空间关系	复制父进程的地址空间（通常为 COW）  独立地址空间，写时复制（Copy-On-Write）	子进程共享父进程地址空间与资源  共享地址空间，子进程修改会影响父进程	由系统/库在内核中创建并执行新程序  不继承地址空间，直接执行目标程序
执行流程 时序与阻塞关系	父子并发执行，不互相阻塞 	子进程先运行，父进程被阻塞（直到子进程 exec/execit） 	父进程不阻塞（可选择等待），子进程独立启动 
资源继承 文件描述符等	继承几乎所有资源（文件描述符、环境变量、信号处理等） 	几乎与父进程完全共享（包括栈、寄存器等）  注意：子进程修改会影响父进程	按需继承（可控制文件描述符、信号等）  可精细控制继承规则
典型用法 适用场景	创建子进程后执行新程序，保留父进程环境 shell 管道 后台服务 多进程服务器	子进程仅执行 exec/execit，不修改数据 exec 新程序 快速替换 避免内存复制	高效启动新程序（如 shell、服务管理、构建工具） shell 启动 服务管理 构建系统
优缺点 核心分析	+ 语义简单，使用灵活 + 父子可并发执行 - 需要复制页表，开销较大（大内存场景）	+ 无需复制地址空间，速度快，开销小 - 父进程被阻塞 - 使用不当风险高（易破坏父进程状态）	+ 高效（内核路径优化） + 可精细控制继承行为 - 依赖系统/库支持（POSIX.1-2008+）



快速选择指南



需要并发执行 + 保留环境

→ 使用 fork()



仅为 exec 新程序 + 追求极致性能

→ 使用 vfork()（谨慎！）



高效启动新程序 + 可控继承

→ 使用 posix_spawn()



注意事项

- 避免在 vfork() 子进程中修改变量或返回
- vfork() 子进程只能调用 _exit() 或 exec()
- posix_spawn() 是现代系统的推荐替代方案

重点 PR 1: vfork / clone 语义修复

这个 PR 修的是父进程等待条件，保证 vfork 类子进程在 exec/exit 前不会让父进程提前继续。

问题

- 旧实现把等待条件错误收窄，部分 CLONE_VFORK 场景没有等待。
- 父进程提前返回会破坏 shell、busybox、cargo 依赖的同步顺序。

修复与测例

- 修复 CLONE_VFORK 等待条件，保证父进程等待 vfork_done。
- 测例：test-vfork。
- 影响范围：BusyBox / shell / cargo 子进程创建路径。

成果：修复进程创建 ABI 语义，使 BusyBox、shell 和 cargo 子进程创建路径具备更稳定的行为基础。

背景 2: futex、robust list 与线程退出

pthread/mutex 的稳定性依赖 futex 快慢路径，以及线程异常退出时内核对遗留锁状态的清理能力。

术语定义

futex 是“用户态内存字 + 内核等待队列”的同步机制；robust list 是线程登记给内核的持锁链表，用于退出时修复 owner died 状态。

用户态快路径

互斥锁、条件变量先用原子指令读写内存字；无竞争时不进入内核，这是 futex 高性能的基础。

内核慢路径

竞争失败时 FUTEX_WAIT 睡眠；释放锁时 FUTEX_WAKE 按地址唤醒等待线程，避免无谓轮询。

robust list

线程持锁退出时，内核扫描登记链表，标记 owner died 并唤醒等待者，避免锁永久卡死。

退出路径容错

链表指针来自用户态，可能为空、坏地址或跨页；清理路径必须把它当作不可信输入处理。

futex 正确性取决于用户态内存字与内核等待队列一致，退出清理还必须安全处理异常用户内存。

重点 PR 2: futex 与线程退出清理

这个 PR 的重点是让线程退出路径能够安全处理坏 robust-list entry 和 pending futex。

问题

- 用户态可以传坏 robust-list 指针，内核不能因此拖垮退出路径。
- pending futex 与普通 entry 混在一起，会让 pthread 退出/回收变得脆弱。

修复与测例

- 坏 entry 容错清理，pending 单独处理。
- 测例：test-futex-robust-list。
- 关联：teardown/context usercopy 修复退出路径。

成果：退出清理路径可以安全处理用户态异常输入，避免单个坏 robust-list 破坏进程回收流程。

背景 3: VFS 与构建型 workload 的文件系统压力

构建系统不会只访问单一文件，它会反复组合路径解析、元数据更新、目录遍历、临时文件替换和设备 I/O。

术语定义

VFS 是内核统一文件系统接口层，屏蔽 tmpfs、ext4/rsext4、设备节点、pipe 等后端差异；应用只看到一组统一 syscall。

VFS 抽象层

负责路径解析、dentry/inode/file 对象和权限生命周期，让 open/read/write/rename 等 syscall 有一致语义。

tmpfs：内存文件系统

主要维护内存中的文件对象和目录树；rename/unlink/open/exec 必须保持同一生命周期视图。

ext4 / rsext4

磁盘文件系统需要管理 inode bitmap、block group、目录项和数据块分配；元数据状态会影响后续创建。

Cargo 真实压力

编译会密集创建小文件、原子 rename、目录扫描、unlink，以及 pipe/device I/O；组合 workload 才是真实检验。

构建型 workload 会组合触发路径解析、元数据分配、rename/unlink 和设备 I/O，检验 VFS 后端一致性。

重点 PR 3: 文件系统与 VFS 正确性

这一组 PR 面向真实应用的文件系统访问模式，补齐 inode 分配、rename/exec 和设备传输语义。

#695 rext4 inode bitmap

ext4 block group 可标记 inode bitmap 未初始化；allocator 需要识别并初始化，再继续分配 inode。

#844 tmpfs rename exec

tmpfs 的目录项和可执行文件生命周期要一致；rename/unlink 不能让后续 exec/open 看到错乱状态。

#800 device transfer

VFS 设备节点读写要遵守完整 transfer 语义，否则 busybox、构建脚本等用户态工具会拿到短读写。

成果：文件系统层面对真实应用的目录项、inode 和设备 I/O 行为更加接近 Linux 预期。

实验 4: StarryOS guest 内自举编译: 单核阶段

单核阶段先把完整 cargo build 跑通; 真正瓶颈是环境、日志、翻译执行和 OS 语义共同拉长反馈。

6h

早期带高频日志的一次完整编译

100min

去掉高频日志后的单核反馈量级

Alpine/riscv64

Docker Linux rootfs; 不同于
macOS host

TCG

riscv64 guest 指令翻译执行, I/O
和 syscall 也走 guest OS

环境差异

不是 macOS 上直接 cargo build, 而是在 Docker 准备的 Linux/Alpine/riscv64 rootfs 中, 让 StarryOS guest 承载 Rust/Cargo workload。

慢的直接原因

QEMU TCG
指令翻译本身慢; 串口/trace 日志会把 trap、syscall、调度和文件系统路径的开销放大到不可调试。

单核阶段找到的 bug

真实编译暴露
fork/exec/wait、futex/robust-list、文件系统、rsex4、tmpfs rename、设备 I/O 和进程回收等问题。

单核阶段的核心价值: 先把 6h 级反馈压到可分析范围, 并把“慢/卡住”拆成可复现的 OS bug。

多核编译性能结果

多核不是只开 `-smp 8`：先缩短编译工作量和 OS 热路径，再比较 guest 内 `jobs=8` 与 macOS host 的差距。

951s -> 331s

guest 端到端调优, 2.87x

422s -> 341s

同 profile 只改 guest jobs, 1.24x

96s -> 42s

macOS native release, 2.29x

85s -> 29s

host aligned reference, 2.93x

速度演进：951s -> 642s -> 515s -> 427s -> 379s -> 341s -> 331s

1. 换底座

从早期 RISC-V/Docker 慢路径转到 AArch64/HVF，本机虚拟化减少翻译开销。

2. 缩短尾段

source/target 放 tmp；关闭 LTO；opt0+cgu256，把 642s 收到 427s。

3. 修 OS 路径

wakeup/runqueue、futex、文件系统等修复后，多核 cargo 尾部能继续推进。

4. 统一口径

331s 是 tuned best；严格 jobs-only 是 422s/341s；host 对照说明 Cargo 本身有并行空间。

下一页开始拆原因：为什么 host 有 2.29x/2.93x，而 guest 同口径只有 1.24x。

性能模型与测试细项

每个瓶颈都用接近 Cargo 行为的微基准拆开验证，而不是只给粗略归因。

$$T_{\text{build}}(N) = T_{\text{std/cache}} + T_{\text{serial}} + T_{\text{parallel}}/N + T_{\text{fs}}(N) + T_{\text{smp}}(N) + T_{\text{wait}}(N)$$

进程链路

fork/exec/wait 1000
次: 20/11/5/3s, 8 路
6.67x。说明 OS
短链路不是完全不能并行。

build-script wave

24 个
build-script: 27/22/27/32/32s
。2 路最快，说明
build.rs/proc-macro
等短阶段过并行会反吃。

FS metadata

rsexrt4 fileio jobs=8: 47.5s ->
14.5s。target 小文件
create/write/rename/unlink
是真实压力。

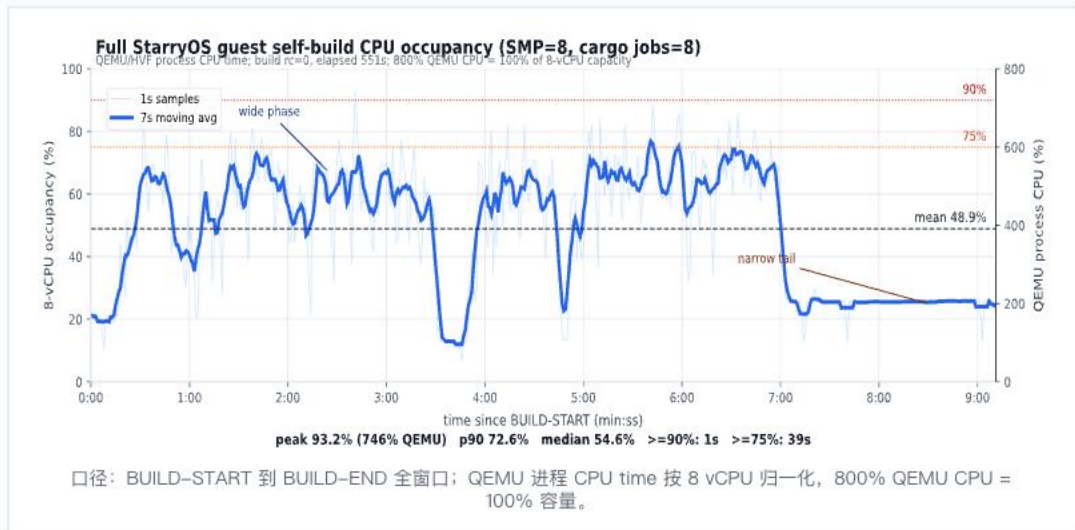
宿主参考

macOS native release
96/42=2.29x; host aligned
85/29=2.93x; guest jobs-only
422/341=1.24x。

模型结论：多核有效，但 full cargo 的收益被 Cargo critical path、长 rustc 窄阶段、build.rs/proc-macro 短阶段和 FS/SMP 共同限制。

为什么 8 核总是打不满

完整 551s 采样显示：8 核 guest 能短时拉高到 93.2%，但均值只有 48.9%，90% 以上仅 1 秒。



证据等级：曲线/日志直接测得；原因由 Cargo timing、host 对照和 guest 短基准共同支撑。

1. 负载宽度阶段性不足

完整曲线只有 39s >=75%，>=90% 只有 1s；多数时间没有 8 个持续可运行编译任务。

2. rustc / 尾段有限并行

约 7:00 后进入窄尾段，QEMU CPU 长时间约 200%，即约 2 个忙 vCPU。

3. 短阶段过并行会反吃

24 个 build-script wave: j1/j2/j4/j6/j8 = 27/22/27/32/32s, 2 路最快。这里指 build.rs/proc-macro, 不指 rustc 本体。

4. OS/FS 路径确实影响

fork/exec 20/11/5/3s 说明多核有效；rsextd4 j8 47.5s -> 14.5s 说明 FS 路径曾是瓶颈。

结论：setting 已生效，8 核能被用起来；打不满主要来自 Cargo/Rust 负载宽度、尾段串行化和 guest OS/FS 短链路成本。

对齐对比：为什么 host 有收益，guest 只有 1.24x

macOS native release 证明 Cargo 本身有并行收益；guest jobs-only 更低，说明差距落在 guest OS/FS/SMP 承载成本上。

96s -> 42s

macOS native release, 2.29x

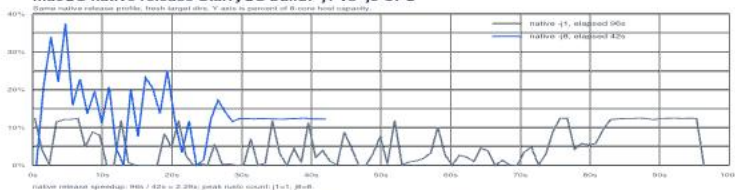
85s -> 29s

host aligned reference, 2.93x

422s -> 341s

guest jobs-only, 1.24x

macOS native release StarryOS build: -j1 vs -j8 CPU



同一 native release profile: -j8 比 -j1 快，但 CPU 仍远低于 8 核常满。

对比结论

Cargo DAG 有可并行空间；但 guest 还要承担
fork/exec、wait/futex、VFS/FS、页分配、调度和 QEMU guest userland 成本。

这自然引出下一页：把慢拆成 Cargo、OS、QEMU 三个边界，而不是把所有问题都说成 QEMU 慢。

成因拆解：Cargo、OS、QEMU 分别承担什么

最后把瓶颈按归属拆清楚，避免把所有慢都归因给 QEMU 或 Cargo。

归属	原因	证据
Cargo DAG / 尾部	jobs=8 是上限，不保证任意时刻有 8 个可运行 rustc。尾段依赖链变窄，额外核心无事可做。	完整 occupancy 图：>=90% 仅 1s；约 7:00 后长期约 2 个忙 vCPU。
guest OS 热路径	fork/exec/wait、pipe、futex、进程回收、页分配和唤醒路径会被 cargo 大量进程和 build.rs/proc-macro 阶段反复触发。	fork/exec/wait 1000 次 20/11/5/3s；RawMutex/wakeup/topology 等 PR 支撑多核前进。
VFS / 文件系统	target-dir 会产生大量小文件、rename、metadata 查询、目录遍历和临时文件替换。	rsextd4 fileio j8 47.5s -> 14.5s；#695/#844/#800 等修复覆盖真实访问模式。
虚拟化边界	RISC-V TCG 翻译慢；AArch64/HVF 缩短纯翻译成本，但 guest syscall/FS/scheduler 仍是真实开销。	早期带日志约 6h；不打日志约 100min；AArch64/HVF 后进入 951s -> 331s 路线。

结论：多核有收益，但收益被 workload 结构和 guest OS 热路径共同限制；这也是后续优化应继续拆的方向。

课程建议与思考：AI 时代的 OS 实验设计

AI 不替代 OS 理解；它更适合把时间花在任务拆解、证据组织、系统验证和真实工程交付上。

1. 训练任务拆解

Agent

能处理长上下文，但前提是目标、仓库、分支、运行模式和验收标准足够清楚。课程可以重点训练如何把庞大 OS 问题拆成可验证的小任务。

2. 更早接触真实系统

在 AI

辅助下，低年级同学也可以参与读代码、跑测试、看日志、修 bug、提 PR；OS 课可以成为工程能力快速成长的入口。

3. 选修课的多元参与

可以区分基础必做与进阶可选。AI 方向同学能把模型能力用于真实 OS 项目，但最终仍要围绕内存、进程、文件系统、syscall、并发和性能。

我的收获：真实 workload 会同时考验 ABI、文件系统、调度、同步和性能；只有证据链完整，PR 和性能结论才站得住。